

SUMMER TRAINING/INTERNSHIP REPORT

(Term Aug-Dec 2026)

Live Deployment

Github: <https://github.com/Priyanshu-Priya/WebRAG>

Frontend (Live Demo): <https://webrag-frontend.onrender.com>

Backend (API Service): <https://webrag-zkg6.onrender.com> Note: The backend service is hosted on an instance that spins down when idle, so it may require up to 50 seconds for a cold start if it has not been accessed recently.

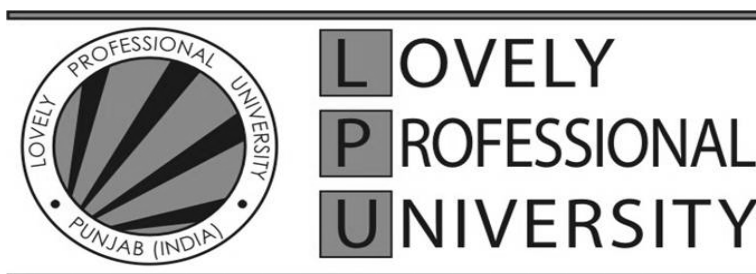
A REPORT ON WEBRAG: A RETRIEVAL-AUGMENTED GENERATION PLATFORM FOR AUTOMATED WEB CRAWLING AND SEMANTIC QA

SUBMITTED BY:

Priyanshu Priya

Registration Number: 12403935

School of Computer Science and Engineering



DECLARATION

I, Priyanshu Priya, hereby declare that the summer training/internship report entitled "WebRAG: A Retrieval-Augmented Generation Platform for Automated Web Crawling and Semantic QA" submitted by me to the Department of Computer Science & Engineering, Lovely Professional University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is an authentic record of my training carried out during the period from June 2026 to July 2026 under the supervision of , Assistant Professor.

The matter embodied in this training report has not been submitted by me for the award of any other degree or diploma of this or any other University.

Date: 14 Jul '26

Place: Hardashpur

Priyanshu Priya

Registration Number: 12403935

CERTIFICATE

This is to certify that the Summer Training/Internship report entitled "WebRAG: A Retrieval-Augmented Generation Platform for Automated Web Crawling and Semantic QA" submitted by Priyanshu Priya (**Registration Number: 12403935**) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Lovely Professional University is a record of student's training work carried out under my supervision.

To the best of my knowledge, the work presented here has reached the requisite standard for the submission of B.Tech summer training report.

[Rohit Bharti and Dipen Saini]

Assistant Professor

Department of Computer Science & Engineering

Head of Department

Department of Computer Science & Engineering

ACKNOWLEDGEMENT

I would like to express my sincere gratitude and deep appreciation to my training supervisor, Rohit Bharti and Dipen Saini, Assistant Professor, Department of Computer Science & Engineering, for providing support, constructive advice, and guidance throughout my summer training curriculum and project execution. Their technical expertise and encouragement have been instrumental in the completion of this project.

I also extend my heartfelt thanks to the Head of the Department and the administration Lovely Professional University for providing the academic infrastructure and laboratories that facilitated this work. I am grateful to the open-source community behind FastAPI, LangChain, and React, whose libraries made this project modular and scalable.

Finally, I would like to thank my parents, family, and peers for their continuous support, patience, and encouragement throughout this academic term.

Priyanshu Priya
Registration Number: 12403935

TABLE OF CONTENTS

Chapter/Title	Page No.
Declaration	i
Certificate	ii
Acknowledgement	iii
Table of Contents	iv
1. INTRODUCTION OF ORGANIZATION	1
1.1 Overview of Retrieval-Augmented Generation (RAG)	1
1.2 Open-Source Ecosystem and Collaborative Development	1
1.3 Project Motivation: Custom Private Knowledge Bases	2
2. SUMMER TRAINING COURSE/INTERNSHIP CONTENT DETAIL	3
2.1 Training Curriculum Overview	3
2.2 Week-by-Week Learning and Execution Path	3
3. SUMMER TRAINING/INTERNSHIP PROJECT DETAIL	6
3.1 Problem Statement	6
3.2 Project Outcomes	6
3.3 System Architecture	7
3.4 Technologies Used	8
4. SOURCE CODE OR SYSTEM SNAPSHOTS	10
4.1 Key Implementation Snippets	10
4.2 System Interface Snapshots	13
12. BIBLIOGRAPHY	15

1. INTRODUCTION OF ORGANIZATION

1.1 OVERVIEW OF RETRIEVAL-AUGMENTED GENERATION (RAG)

Retrieval-Augmented Generation (RAG) represents a foundational shift in how organizations leverage Artificial Intelligence and Large Language Models (LLMs). While baseline LLMs exhibit extraordinary reasoning capabilities, they are inherently limited by their training cutoff dates and a tendency to hallucinate plausible-sounding but false information when queried about niche or internal corporate knowledge. RAG resolves this constraint by decoupling knowledge retrieval from linguistic synthesis. In a RAG pipeline, when a user asks a question, the system queries a search engine or vector database to retrieve relevant textual passages, merges these passages into a structured prompt, and passes them to the LLM. The model is instructed to answer the question grounded strictly in the retrieved text. This approach ensures factual accuracy, data security, and verifiable provenance.

1.2 OPEN-SOURCE ECOSYSTEM AND COLLABORATIVE DEVELOPMENT

This internship and research work was conducted within the framework of an Open-Source Software Development Lab environment, focusing on modern collaborative practices. The development pipeline simulated an industry-grade agile environment, involving version control (Git), code documentation, and incremental integration. The research objectives centered on creating local, low-footprint, private pipelines to crawl, index, and search local networks and intranets. Through this model, students get hands-on experience in full-stack development, ranging from asynchronous Python web crawlers and vector embeddings to reactive web dashboards. Collaborating in this open-source paradigm enforces standard coding practices, automated code formatting, and reproducible execution using containerization technologies such as Docker.

1.3 PROJECT MOTIVATION: CUSTOM PRIVATE KNOWLEDGE BASES

Organizations face a double-edged sword when integrating public LLM models: they wish to benefit from AI productivity, yet they cannot risk transmitting sensitive intellectual property, proprietary source code, or customer records to external servers. Furthermore, public search engines are often slow to index internal corporate websites

or documentation portals, making online search integrations ineffective. The motivation behind WebRAG is to build a self-contained, enterprise-grade, low-overhead RAG application. By executing embeddings local to the host processor and utilizing secure, speed-optimized external API endpoints (such as Groq using speculative decoding), WebRAG provides real-time information retrieval. It automatically crawls configured URLs, cleans noise data (navbars, scripts, templates), and updates database tables only when web content changes, reducing computational waste and storage footprint.

2. SUMMER TRAINING COURSE/INTERNSHIP CONTENT DETAIL

2.1 TRAINING CURRICULUM OVERVIEW

The six-week summer training curriculum was structured to cover the entire lifecycle of a modern AI-powered web application. The syllabus advanced from python asynchronous coding paradigms and web scraper architecture to natural language processing (NLP), semantic search indexing, relational database schema designing, RESTful API routing, real-time communication protocols, frontend reactive states, and DevOps deployment. Each week, specific technical milestones were defined and verified against the WebRAG project codebase.

2.2 WEEK-BY-WEEK LEARNING AND EXECUTION PATH

- **Week 1: Foundations of Asynchronous Scrape & Crawl Architectures.**
Acquired deep proficiency in Python's asynchronous programming framework (`asyncio`). Studied the mechanics of the HTTP/HTTPS protocols, including request headers, cookies, query parameters, redirect cycles, and rate-limiting. Built concurrent crawler routines utilizing `httpx` for standard websites. Studied robots.txt policies and designed a non-blocking scheduler to respect crawl delays.
- **Week 2: DOM Parsing, HTML Cleaning, and Text Chunking.** Focused on document object model (DOM) parsing. Used `BeautifulSoup` to strip boilerplate tags (e.g., `<script>`, `<style>`, `<nav>`, `<footer>`) from raw HTML, leaving only semantic text. Designed character-based chunking algorithms to slice long passages into overlapping segments (typically 1000 characters with 10% overlap). Implemented SHA-256 hash algorithms on raw texts to track page states and detect changes.
- **Week 3: Vector Embeddings and Indexing with Vector Databases.**
Explored vector space models and token embeddings. Integrated local embedding models (such as BAAI/bge-base-en-v1.5 sentence-transformers) and cloud APIs (OpenAI text-embedding-3-small). Learned how to configure and interact with ChromaDB, a lightweight vector database. Programmed vector collection creation, document insertion with chunk IDs, metadata filtering, and L2 distance similarity searching.

- Week 4: API Development, Relational Databases, and Websockets.**
 Designed relational database schemas using `SQLAlchemy` (ORM) and SQLite, creating tables for Collections, URLs, Pages, Chunks, Changes, History, and Settings. Built REST APIs using `FastAPI` to execute CRUD operations on collections. Integrated real-time WebSocket connection managers to stream crawling progress logs directly to clients. Programmed background timers using `APScheduler` to run periodic page freshness checks.
- Week 5: Retrieval-Augmented Generation (RAG) and LLM Orchestration.** Learned prompt engineering methodologies to enforce strict grounded answering, preventing model hallucinations. Configured the LangChain ecosystem to link ChromaDB search results with Groq's high-speed API (`llama-3.3-70b-specdec`). Engineered conversational memory that appends multi-turn history. Programmed a parsing layer that automatically maps retrieved chunk origins to inline numerical footnotes (e.g. [1], [2]) in the LLM's response.
- Week 6: Frontend Development, WebSocket Integration, and Docker Deployment.** Created an interactive frontend using React (v19) and Vite. Styled pages (Dashboard, Collections, Chat, Settings) using TailwindCSS, and added micro-animations with Framer Motion. Connected frontend inputs to backend REST and WebSocket APIs. Wrote Dockerfiles and `docker-compose.yml` to bundle the database, vector store, backend server, and Nginx frontend server into a unified deployment script.

3. SUMMER TRAINING/INTERNSHIP PROJECT DETAIL

3.1 PROBLEM STATEMENT

Modern enterprise websites and documentation platforms are dynamic, client-side rendered, and contain vast amounts of unstructured text. Traditional search tools and naive Large Language Model agents suffer from several critical engineering limitations:

1. **Dynamic Client-Side Pages:** Simple HTTP request scrapers fail to extract text from modern web apps built with React, Angular, or Vue because they only fetch empty HTML headers without executing the Javascript that mounts the text.
2. **Redundant Embedding Overheads:** Re-embedding and re-indexing entire websites on every sync cycle is computationally expensive, leads to high cloud API usage fees, and locks local databases during index updates.
3. **Lack of Auditability:** Standard chatbot interfaces provide answers without citing source paragraphs. If an AI agent gives instructions to a user, the user has no way of verifying the document chunk from which the information was derived, posing significant compliance risks.
4. **Hallucinations:** Without strict prompt constraints and semantic scoping, LLMs tend to extrapolate beyond their retrieved context, mixing training data with local database details.

3.2 PROJECT OUTCOMES

The development of WebRAG successfully addressed these engineering issues. The system achieved the following outcomes:

1. **Dual-Scraping Engine:** Programmed a crawler that initiates using fast asynchronous HTTPX requests and automatically falls back to Playwright headless browser rendering if the page contents contain dynamic script elements.
2. **Incremental SHA-256 Indexing:** Built a content hash detector. During crawls, WebRAG computes a SHA-256 hash of the cleaned text. If the hash matches the SQLite record, the page is skipped. If a change is detected, the old page chunks are deleted from ChromaDB and SQLite, and only the new page chunks are embedded, saving significant runtime.
3. **Detailed Content Diffing:** Integrated Python's `'difflib'` library. When a page is

updated, the change detector generates a line-by-line diff report displaying exactly what text was added, removed, or changed. This report is persisted in the database, allowing administrators to audit site modifications.

4. High-Fidelity Citations: Structured a customized system prompt that forces the Groq LLM to append numbered brackets matching the retrieved document chunk index. If the retrieved context cannot satisfy the user question, the model responds with a standardized fallback sentence: "I couldn't find information about this in the indexed websites."

5. Interactive Dashboard: Implemented a responsive, glassmorphic React dashboard with collection setups, vector settings controls, dynamic search lists, conversational chat widgets, and real-time streaming progress logs.

3.3 SYSTEM ARCHITECTURE

The system architecture follows a decoupled, three-tier model comprising a React frontend client, a FastAPI backend server, and a persistence layer composed of SQLite (relational) and ChromaDB (vector) databases. The workflow triggers when a collection is created. The crawler scans the seed URLs, parses the DOM trees, passes clean text to the embedding manager, and indexes the resulting vectors in ChromaDB. When a user executes a chat query, the FastAPI backend retrieves the top 'K' most similar chunks from ChromaDB, formats the chat history, and packages them into a prompt for the ChatGroq model. The diagram below (Figure 1) illustrates the data flow and integration of components.

3.4 TECHNOLOGIES USED

The core technologies, libraries, and frameworks utilized to build WebRAG are organized in Table 1 below, describing their specific role in the system architecture.

Table 1: Technologies Used and Their Functions in WebRAG System

Technology Layer	Library/Framework	Role in WebRAG Architecture
Backend Framework	FastAPI / Uvicorn	Handles HTTP REST endpoints, WebSocket routing, and CORS middlewares.

Web Scraper Engine	Playwright / HTTPX	Fetches pages asynchronously; Playwright handles JS dynamic mounting.
HTML Parser	BeautifulSoup4	Cleans raw HTML by stripping scripting tags, navbar, footer, and styling elements.
Database Mapper	SQLAlchemy ORM	Manages database connections, schemas mapping, and transactional operations.
Relational DB	SQLite	Persists collections, crawls metadata, SHA-256 hashes, diff reports, and chat history.
Vector Store	ChromaDB	Maintains high-dimensional document chunk vector indexes and handles similarity searches.
LLM Orchestrator	LangChain Core/Community	Integrates RAG components, wraps model APIs, and manages prompts/memory.
Embedding Engine	huggingface_hub API / BGE	Converts textual chunks into dense numerical embedding vectors.
LLM API Provider	Groq Cloud API (Llama 3.3)	Executes fast grounded question answering under strict prompt controls.
Frontend UI Client	React (v19) / Vite / Axios	Renders the dashboard UI, handles component states, and communicates via HTTP/WS.

4. SOURCE CODE OR SYSTEM SNAPSHOTS

4.1 KEY IMPLEMENTATION SNIPPETS

This section contains key programming snippets from the WebRAG implementation. The backend is written in Python, conforming to FastAPI schemas and LangChain guidelines. The code uses asynchronous loops to manage high-throughput crawling.

4.1.1 ASYNCHRONOUS SCRAPER WITH DYNAMIC BROWSER FALLBACK

The following snippet from `crawler.py` demonstrates the fetch sequence. The system attempts a fast HTTP get request using `httpx` first. If the page is detected to contain client-side JavaScript templates or if headers block raw GETs, the scraper spawns a headless Chromium browser using `playwright` to wait until the DOM mounts.

```
async def crawl_url(self, url: str) -> str:
    # Try fast HTTP request first
    try:
        async with httpx.AsyncClient() as client:
            status, text, headers = await
self._fetch_with_httpx(client, url)
            if status == 200 and not
self._is_js_dependent(text):
                return text
        except Exception as e:
            logger.warning(f"HTTPX request failed for {url}:
{e}. Trying Playwright...")

        # Fallback to headless Playwright browser rendering
        if HAS_PLAYWRIGHT:
            try:
                return await self._fetch_with_playwright(url)
            except Exception as e:
                logger.error(f"Playwright browser fallback
failed: {e}")
                raise e
            raise Exception("Scraping failed: HTML content was
unreadable and Playwright not available.")
```

4.1.2 HASH-BASED CHANGE DETECTOR AND TEXT DIFFING

The `change\_detector.py` uses Python's standard `difflib` library. By comparing text sentences, it calculates changes line-by-line and counts added/removed strings. The change report is stored in SQLite.

```
def detect_changes(old_text: str, new_text: str) ->
Dict[str, Any]:
    if not old_text:
        return {"sections_added":
len(new_text.splitlines()), "sections_removed": 0,
"report_text": "First index."}

    old_lines = [line.strip() for line in
old_text.split(".") if line.strip()]
```

```

    new_lines = [line.strip() for line in
new_text.split(".") if line.strip()]
    diff = [line for line in difflib.ndiff(old_lines,
new_lines) if not line.startswith("?")]

    added, removed, changed = 0, 0, 0
    idx, n = 0, len(diff)
    while idx < n:
        tag, content = diff[idx][0], diff[idx][2:]
        if tag == "+":
            added += 1
            idx += 1
        elif tag == "-":
            if idx + 1 < n and diff[idx + 1][0] == "+":
                changed += 1
                idx += 2
            else:
                removed += 1
                idx += 1
        else:
            idx += 1
    return {"sections_added": added, "sections_removed":
removed, "paragraphs_changed": changed}

```

4.1.3 GROUNDED QA CHAIN AND SOURCE CITATION PROMPT

The Retrieval-Augmented Generation logic from `qa\_chain.py` structures a strict prompt forcing the Llama-3 model on Groq to cite its sources using numbered brackets, and fallback safely if the information is not present.

```

PROMPT_TEMPLATE = """You are WebRAG, a professional AI
assistant.
You must answer the user's question using ONLY the
retrieved web page context below.
Rules:
1. Use ONLY the provided Context to answer the question.
2. If the Context does not contain enough info, reply
exactly: "I couldn't find information..."
3. Cite sources using numbered bracket inline footnotes at
the end of the sentence (e.g. [1], [2]).
Conversation History:
{history}
Context:
{context}
Question: {question}
Answer: """

def answer_question(self, collection_id: int, question:
str, history_turns=None) -> Dict[str, Any]:
    matched_docs_with_scores =
self.chroma_manager.similarity_search(collection_id,
question, k=5)
    context_str = ""
    for idx, doc in enumerate(matched_docs_with_scores):
        context_str += f"--- Document Chunk {idx+1}
(Source: {doc.metadata.get('url')}) ---\n"
        context_str += f"Content: {doc.page_content}\n\n"

    formatted_prompt =
self.prompt_template.format(context=context_str,
question=question, history=history_str)
    response = self.llm.invoke(formatted_prompt)

```

```

return {"answer": response.content, "sources":
self.compile_citations(matched_docs_with_scores)}

```

4.2 SYSTEM INTERFACE SNAPSHOTS

The following diagram represents the core pipeline execution flow and system components of WebRAG. It visualizes the integration between the crawler, database stores, embedding generators, vector namespaces, and Groq LLM pipelines.

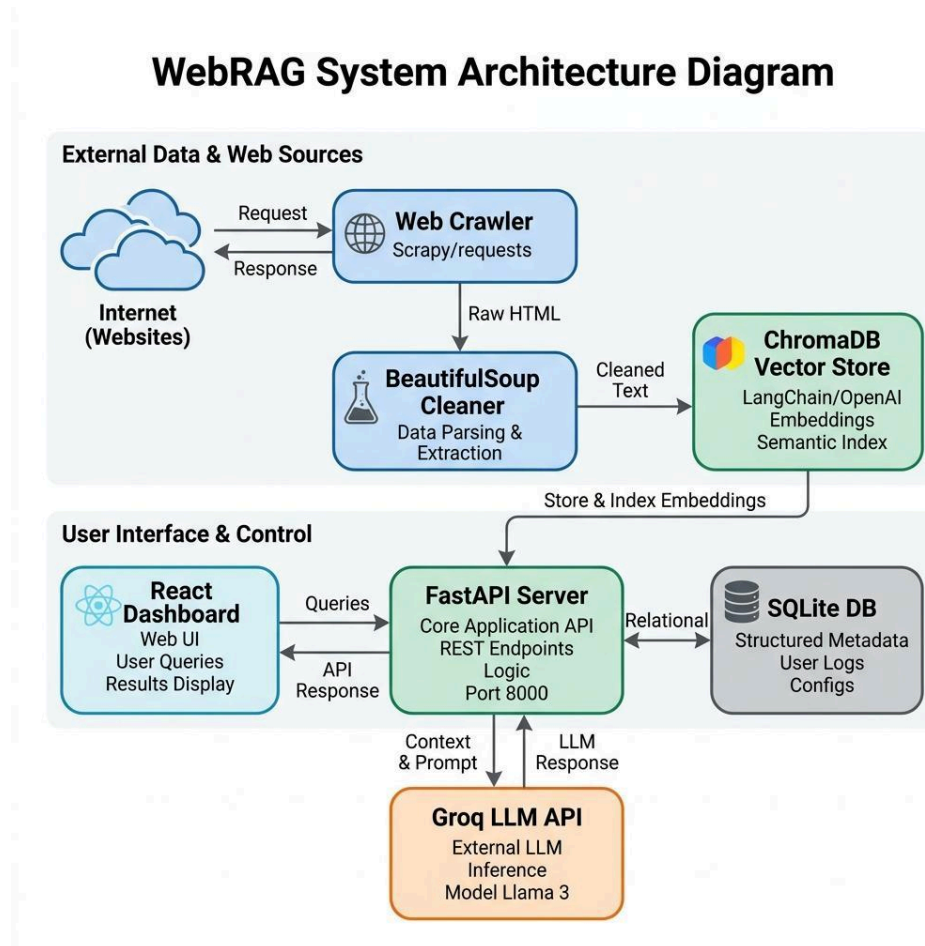


Figure 1: WebRAG System Architecture and Pipeline Execution Flow

12. BIBLIOGRAPHY

This bibliography lists the documentation guidelines, frameworks, and scholarly literature referenced during the research, design, and deployment of the WebRAG Retrieval-Augmented Generation application.

- FastAPI Web Framework Documentation. <https://fastapi.tiangolo.com/> - Referenced for designing RESTful API controllers and CORS middlewares.
- LangChain Documentation and Conceptual Guidelines. <https://python.langchain.com/> - Utilized for building prompt templates, retrieval chains, and LLM integrations.
- Chroma DB: The AI-native open-source vector database. <https://www.trychroma.com/> - Consulted for semantic similarity search interfaces and vector namespace management.
- Playwright API for Python. <https://playwright.dev/python/> - Referenced for launching headless chromium instances to scrap client-side Javascript dynamic objects.
- BeautifulSoup4: Python HTML/XML parser library. <https://www.crummy.com/software/BeautifulSoup/bs4/> - Utilized for DOM node stripping and boilerplate noise cleaning.
- Groq Cloud LLM Developer Documentation. <https://console.groq.com/docs> - Consulted for llama-3 inference speeds, chat completions endpoints, and temperature setting optimization.
- React Core Documentation. <https://react.dev/> - Referenced for reactive state hooks, client component layout routing, and WebSocket hook subscriptions.
- BGE Embeddings Model: BAAI/bge-base-en-v1.5 open source model. HuggingFace Hub - Utilized for local feature extraction to yield 768-dimensional dense vectors.